Clumpy Medium Extension of LaRT: Algorithm Description

Contents

1. Overview **1**

2. Physical Setup **1**

2.1 Geometry . . . 1

2.2 Clump Count . . . 2

2.3 Clump Physical Parameters . . . 2

2.4 Clump Velocities . . . 3

3. Clump Placement: Random Sequential Addition (RSA) **3**

3.1 Concept . . . 3

3.2 Uniform Sampling Inside the Sphere . . . 4

3.3 Placement Algorithm . . . 4

3.4 Linked-List Hash Grid and the 27-Cell Rule . . . 5

3.5 Acceptance Rate and Jamming Limit . . . 6

4. CSR Acceleration Grid **6**

4.1 Motivation . . . 6

4.2 Data Structure: Compressed Sparse Row . . . 6

4.3 Cell Grid and Domain . . . 7

4.4 Clump Registration: AABB Overlap Test . . . 7

4.5 Two-Pass Construction . . . 7

5. Raytrace Through the Clump Medium **9**

5.1 Overview . . . 9

5.2 Ray–Sphere Intersection . . . 9

5.3 DDA Search for Next Clump . . . 9

5.4 Optical Depth Traversal . . . 10

5.5 Optical Depth to Edge . . . 11

6. Peel-Off Observations **11**

7. FITS Output of Clump Information **11**

8. Input Parameters Summary **13**

9. Memory Layout **14**

10. Validation and Examples **14**

1. Overview

The clumpy medium mode adds a population of N non-overlapping spherical clumps inside a spherical domain of radius R. Each clump is a sphere of radius r_c with uniform Lyman-α opacity; the inter-clump medium is vacuum.

The mode is activated by

```
par%use_clump_medium = .true.
```

in the input namelist. All raytrace procedure pointers are redirected to clump-aware routines (`raytrace_clump.f90`) while scatter and peel-off reuse the standard Cartesian routines from `scattering_car.f90` and `peelingoff_rect.f90`, since the uniform clump physical parameters (T, $v_{\rm bulk}$) are broadcast to the Cartesian grid arrays (`grid%Dfreq`, `grid%voigt_a`).

New source files introduced for this mode:

File	Role
`clump_mod.f90`	Clump population: init, RSA placement, CSR grid, raytrace helpers
`raytrace_clump.f90`	Raytrace through the clump medium (DDA + sphere intersection)
`grid_mod_clump.f90`	Grid setup: creates Cartesian shell grid, then calls `init_clumps`

2. Physical Setup

2.1 Geometry

The outer domain is a sphere of radius R = `par%rmax` (code units). N spherical clumps of radius r_c = `par%clump_radius` are placed at random inside this sphere (non-overlapping).

A Cartesian grid of $N_x \times N_y \times N_z$ cells is created with the same bounding box $[-R, R]^3$. Grid arrays (`rhokap`, `Dfreq`, `voigt_a`, `vf`) are set to uniform values:

- `rhokap` $= 0$ everywhere (opacity lives only inside clumps);
- `Dfreq` $= \Delta\nu_D^{cl}$ (clump Doppler frequency);
- `voigt_a` $= a_V^{cl}$ (clump Voigt parameter);
- `vfx` $=$ `vfy` $=$ `vfz` $= 0$.

Peel-off and scattering routines read these uniform grid values, so no special-casing is needed inside those modules.

2.2 Clump Count

The user specifies one of three equivalent parameters; the code derives N from whichever is set:

From $f_{\rm cov}$ (covering factor): The mean number of clumps intercepted by a radial ray from the centre to the sphere surface is

$$f_{\rm cov} = N\,\frac{\pi r_c^2}{\pi R^2}\cdot\frac{R}{\frac{4}{3}\pi R^3/(4\pi R^2)} = \frac{3}{4}\,N\Big(\frac{r_c}{R}\Big)^2, \tag{1}$$

hence

$$N = \frac{4}{3}\,f_{\rm cov}\left(\frac{R}{r_c}\right)^2. \tag{2}$$

Input parameter: `par%clump_f_cov`.

From $f_{\rm vol}$ (volume filling factor):

$$f_{\rm vol} = N\left(\frac{r_c}{R}\right)^3, \qquad N = f_{\rm vol}\left(\frac{R}{r_c}\right)^3. \tag{3}$$

Input parameter: `par%clump_f_vol`.

Direct specification: `par%clump_N_clumps`.

2.3 Clump Physical Parameters

The clump temperature T_c defaults to `par%temperature` unless `par%clump_temperature` > 0. Derived quantities:

$$v_{\rm th} = v_{\rm th,1}\sqrt{T_c}, \quad v_{\rm th,1} = \sqrt{2k_B/m_H} \approx 12.84\ {\rm km\,s^{-1}}/\sqrt{10^4\,{\rm K}}, \tag{4}$$

$$\Delta\nu_D^{cl} = v_{\rm th}/(\lambda_0 \times {\rm km2um}), \tag{5}$$

$$a_V^{cl} = \frac{A_{21}}{4\pi\,\Delta\nu_D^{cl}}, \tag{6}$$

where λ_0 is the Lyman-α rest wavelength.

The line-centre optical depth of a clump (centre to surface) is `par%clump_tau0`; from this the uniform opacity is

$$\kappa_c = \frac{\tau_0}{\phi(0, a_V^{cl})\,r_c}, \tag{7}$$

where $\phi(x, a)$ is the Voigt profile at dimensionless frequency x. Alternatively, the hydrogen number density `par%clump_nH` may be given, in which case $\kappa_c = n_H\,\sigma_0\,d_{\rm cm}$ (requires `par%distance_unit`).

2.4 Clump Velocities

Each clump's velocity is the sum of two independent components:

$$\mathbf{v}_i = \mathbf{v}_{\rm sys}(\mathbf{r}_i) + \mathbf{v}_{\rm rand,i}, \tag{8}$$

where $\mathbf{v}_{\rm sys}$ is a position-dependent systematic flow (set by `par%velocity_type`) and $\mathbf{v}_{\rm rand}$ is an independent Gaussian random component (set by `par%clump_sigma_v`). Either component can be zero.

Random component: If `par%clump_sigma_v` > 0, each clump is assigned $\mathbf{v}_{{\rm rand},i} \sim \mathcal{N}(0, \sigma_v^2)$ per component (drawn in `generate_clumps`).

Systematic component (`par%velocity_type`): After RSA placement, `assign_clump_velocities_from_type` evaluates the flow velocity at each clump centre and *adds* it to the existing (possibly non-zero) velocity. Supported types:

`velocity_type`	Formula	Parameters
`'hubble'`	$\mathbf{v} = V_{\rm exp}\,\mathbf{r}/R$	`Vexp`
`'constant_radial'`	$\mathbf{v} = V_{\rm exp}\,\hat{\mathbf{r}}$	`Vexp`
`'parallel_velocity'`	$\mathbf{v} = (V_x, V_y, V_z)$	`Vx, Vy, Vz`
`'ssh'`	see below	`Vpeak, rpeak, DeltaV`
`'rotating_solid_body'`	$\mathbf{v} = V_{\rm rot}(-y, x, 0)/R$	`Vrot`
`'rotating_galaxy_halo'`	flat rotation about z	`Vrot, rinner`

All velocity magnitudes are in km/s. The `'ssh'` model (Song, Seon & Hwang 2020) gives a linear rise inside $r_{\rm peak}$ and a Hubble-like outer flow:

$$v(r) = \begin{cases} V_{\rm peak}\, r/r_{\rm peak} & r < r_{\rm peak}, \\ V_{\rm peak} + \Delta V\,(r - r_{\rm peak})/(R - r_{\rm peak}) & r \geq r_{\rm peak}. \end{cases} \tag{9}$$

The `'rotating_galaxy_halo'` model uses the cylindrical radius $\rho = \sqrt{x^2 + y^2}$ and gives solid-body rotation inside $r_{\rm inner}$ and a flat curve ($v_\phi = V_{\rm rot}$) outside.

3. Clump Placement: Random Sequential Addition (RSA)

3.1 Concept

Random Sequential Addition (RSA) is a simple stochastic algorithm for generating configurations of non-overlapping hard spheres. Candidate positions are drawn at random one at a time; each is tested against all previously accepted clumps and either accepted (no overlap) or discarded irreversibly (overlap detected). Because accepted positions are never moved, RSA requires no iterative relaxation.

The non-overlap condition enforces a minimum centre-to-centre distance of $2r_c$. RSA reliably converges for volume filling factors $f_{\rm vol} \lesssim 35\%$; above this threshold, the fraction of unblocked placement space decreases so steeply that placing the last few clumps may require enormous numbers of attempts. The theoretical 3-D jamming limit for RSA is $f_{\rm vol} \approx 38\%$.

3.2 Uniform Sampling Inside the Sphere

Candidate positions must be distributed *uniformly* inside the outer sphere of radius R. This is achieved by box-rejection:

1. Draw x_c, y_c, z_c each uniformly from $[-R, R]$.
2. Accept if $x_c^2 + y_c^2 + z_c^2 \le R^2$; otherwise draw again.

The acceptance probability equals the sphere-to-cube volume ratio $\pi/6 \approx 52\%$, so on average only $6/\pi \approx 1.91$ draws are needed per accepted point. Box-rejection avoids trigonometric functions and yields an exactly uniform distribution in 3-D Cartesian space.

3.3 Placement Algorithm

```
do while (icl < N_clumps)

  !--- Step 1: draw candidate inside sphere (box rejection)
  do
    xc = (2.0*rand() - 1.0) * R
    yc = (2.0*rand() - 1.0) * R
    zc = (2.0*rand() - 1.0) * R
    if (xc**2 + yc**2 + zc**2 <= R**2) exit
  enddo

  !--- Step 2: locate candidate in hash-grid cell (ig, jg, kg)
  ig = int((xc - xmin) / cell_size)
  jg = int((yc - ymin) / cell_size)
  kg = int((zc - zmin) / cell_size)

  !--- Step 3: check 27 neighbouring cells for overlap
  overlap = .false.
  outer: do kg2 = kg-1, kg+1
         do jg2 = jg-1, jg+1
         do ig2 = ig-1, ig+1
    if (out of bounds) cycle
```

```
    jnb = head(ig2, jg2, kg2)        ! first clump in this hash cell
    do while (jnb > 0)
      if (dist2(xc,yc,zc, cl_x(jnb),cl_y(jnb),cl_z(jnb)) < (2*rc)**2) then
        overlap = .true.; exit outer ! reject immediately on first overlap
      endif
      jnb = nxt(jnb)                 ! follow linked list to next clump
    enddo
  enddo; enddo; enddo outer

  if (overlap) cycle                 ! reject: draw new candidate

  !--- Step 4: accept -- record position and random velocity
  icl = icl + 1
  cl_x(icl) = xc;  cl_y(icl) = yc;  cl_z(icl) = zc
  if (sigma_v > 0.0) then
    cl_vx(icl) = sigma_v * gauss_random()  ! one independent draw per component
    cl_vy(icl) = sigma_v * gauss_random()
    cl_vz(icl) = sigma_v * gauss_random()
  endif

  !--- Step 5: prepend clump to the linked list for cell (ig, jg, kg)
  nxt(icl)          = head(ig, jg, kg)
  head(ig, jg, kg) = icl

enddo
```

3.4 Linked-List Hash Grid and the 27-Cell Rule

Naively, checking candidate n against all $n-1$ accepted clumps costs $O(n)$, giving $O(N^2)$ total work. The linked-list hash grid reduces each overlap check to $O(1)$ expected cost by spatially indexing accepted clumps.

The domain $[-R, R]^3$ is partitioned into N_g^3 uniform cells with

$$N_g = \min\bigl(512,\ \lfloor N^{1/3} \rfloor + 1\bigr), \qquad \ell_{\mathrm{RSA}} = \frac{2R}{N_g}. \tag{10}$$

Two integer arrays encode the linked lists: `head(i,j,k)` is the index of the first clump in cell (i, j, k) (0 if empty), and `nxt(icl)` is the next clump in the same cell list (0 = end of list). When a new clump is accepted (Step 5 above), it is prepended to the head of its cell's list in $O(1)$.

Why 27 cells are sufficient. Two clumps overlap iff their centre separation is $< 2r_c$. If the candidate lies in cell (i, j, k), any overlapping clump must satisfy $|x_{\rm cl} - x_c| < 2r_c$. Since $\ell_{\rm RSA} \geq 2r_c$ by construction (eq. 10), the overlapping clump's cell index i' satisfies $i' \in \{i-1, i, i+1\}$, and likewise for j' and k'. Checking all $3^3 = 27$ cells centred on (i, j, k) is therefore both sufficient and complete — no overlapping clump can lie outside this neighbourhood.

Cell-size constraint. The 27-cell rule requires $\ell_{\rm RSA} \geq 2r_c$, i.e. $N_g \leq R/r_c$. For the reference case ($R = 1$, $r_c = 0.01$): $R/r_c = 100$, $N_g = \lfloor 66667^{1/3} \rfloor + 1 = 42$, $\ell_{\rm RSA} = 2/42 \approx 0.048 \gg 2r_c = 0.02$. ✓

3.5 Acceptance Rate and Jamming Limit

As more clumps are placed, their exclusion zones (spheres of radius $2r_c$ around each accepted clump) collectively block an increasing fraction of the placement domain. The per-candidate acceptance probability $p_{\rm acc}$ decreases monotonically with $f_{\rm vol}$ and approaches zero at the jamming limit.
For the reference case ($f_{\rm cov} = 5$, $r_c/R = 0.01$, $N = 66667$, $f_{\rm vol} = 0.067$), the overall acceptance rate is $\approx 74\%$: roughly 90 000 candidates are drawn to place 66 667 clumps. The algorithm remains practical for $f_{\rm vol} \lesssim 35\%$.

4. CSR Acceleration Grid

4.1 Motivation

During radiative transfer, a photon in vacuum must find the nearest clump along its current flight direction at every propagation step. Checking all N clumps per step would cost $O(N)$ per query and become prohibitive for large populations ($N \sim 10^4$–10^5). The CSR acceleration grid pre-sorts clumps into spatial cells; the DDA traversal (§5.3) then visits only the cells the ray crosses, and within each cell tests only the few registered clumps. Together, DDA+CSR achieves $O(1)$ amortised cost per nearest-clump query, independent of N.

4.2 Data Structure: Compressed Sparse Row

Instead of a list-of-lists (which requires many small dynamic allocations), all per-cell clump lists are packed into two flat integer arrays in *Compressed Sparse Row* (CSR) format — the same layout used for sparse matrices:

- `cg_start(0:Ncell)`: `cg_start(icell)` is the 1-based index of the first entry for cell `icell` in `cg_list`. The entries for cell `icell` occupy positions `cg_start(icell) ... cg_start(icell+1)-1`; their count is `cg_start(icell+1) - cg_start(icell)`.
- `cg_list(1:Ntot)`: flat concatenation of all per-cell clump-index lists.

Retrieving the clump list for any cell is an $O(1)$ array slice with no pointer chasing. To illustrate: suppose four cells contain the clumps $\{2,5\}$, $\{\}$, $\{1,3\}$, $\{4\}$ respectively:

$$\texttt{cg_start} = [1,\ 3,\ 3,\ 5,\ 6],$$
$$\texttt{cg_list} = [2,\ 5,\ 1,\ 3,\ 4].$$

The clumps in cell 2 are `cg_list(3:4)` $= \{1,3\}$. Both arrays reside in MPI-3 shared memory (one copy per compute node).

4.3 Cell Grid and Domain

The CSR grid reuses the same cell count $M = N_g$ as the RSA hash grid (§3.4), but covers a slightly larger domain $[-R - r_c,\ R + r_c]^3$ with uniform cell size

$$\delta = \frac{2(R + r_c)}{M}. \tag{11}$$

Expanding the domain by r_c in each direction ensures two properties:

1. A clump centred exactly on the outer sphere surface ($|\mathbf{c}| = R$) fits entirely inside the grid.
2. A photon entering from outside the outer sphere starts inside a valid grid cell, so the DDA traversal can begin immediately.

4.4 Clump Registration: AABB Overlap Test

A clump of radius r_c centred at (c_x, c_y, c_z) is registered in every cell whose axis-aligned bounding box (AABB) overlaps the clump sphere. Cell (i, j, k) spans $[x_{\min} + i\,\delta,\ x_{\min} + (i+1)\,\delta]$ along x. The range of cell indices that overlap $[c_x - r_c,\ c_x + r_c]$ is

$$i \in \left[\left\lfloor \frac{c_x - r_c - x_{\min}}{\delta} \right\rfloor,\ \left\lfloor \frac{c_x + r_c - x_{\min}}{\delta} \right\rfloor\right], \tag{12}$$

and likewise for j and k. The clump is registered in every cell in the Cartesian product of these three index ranges.

For the reference case ($r_c = 0.01$, $\delta \approx 0.048$, so $r_c/\delta \approx 0.21 < 1$), each clump's index range spans at most 2 values per dimension, so a clump touches at most $2^3 = 8$ cells. The total registration count satisfies $N \le N_{\text{tot}} \le 8N$.

4.5 Two-Pass Construction

The CSR arrays are built in two sequential passes over all N clumps, avoiding any intermediate list storage (the standard technique for CSR sparse-matrix assembly):

Pass 1 (count): Compute the cell ranges (eq. 12) for each clump and increment a counter `cnt(icell)` for every touched cell. Then compute the exclusive prefix sum:

$$\texttt{cg_start(0)} = 1, \qquad \texttt{cg_start(icell+1)} = \texttt{cg_start(icell)} + \texttt{cnt(icell)}.$$

Allocate `cg_list` of length $N_{\text{tot}} = \texttt{cg_start(Ncell)} - 1$ as an MPI-3 shared-memory window.

Pass 2 (fill): Repeat the same cell-range loop. For each touched cell, write the clump index at write pointer `ptr(icell)`, then advance the pointer by one. Initialise `ptr(icell) = cg_start(icell)` before this pass.

```
!--- Pass 1: count registrations per cell
cnt(:) = 0
do icl = 1, N_clumps
  i0 = max(0,   floor((cl_x(icl) - rc - xmin) / delta))
  i1 = min(M-1, floor((cl_x(icl) + rc - xmin) / delta))
  j0 = ...; j1 = ...                    ! analogous for y
  k0 = ...; k1 = ...                    ! analogous for z
  do k = k0,k1;  do j = j0,j1;  do i = i0,i1
    cnt(cell_idx(i,j,k)) = cnt(cell_idx(i,j,k)) + 1
  enddo;  enddo;  enddo
enddo
! exclusive prefix sum -> cg_start
cg_start(0) = 1
do icell = 0, Ncell-1
  cg_start(icell+1) = cg_start(icell) + cnt(icell)
enddo
Ntot = cg_start(Ncell) - 1
! allocate cg_list(Ntot) as MPI-3 shared-memory window

!--- Pass 2: fill cg_list
ptr(0:Ncell-1) = cg_start(0:Ncell-1)   ! one write pointer per cell
do icl = 1, N_clumps
  ... (same i0,i1,j0,j1,k0,k1 as Pass 1)
  do k = k0,k1;  do j = j0,j1;  do i = i0,i1
    ic = cell_idx(i,j,k)
    cg_list(ptr(ic)) = icl
    ptr(ic) = ptr(ic) + 1
  enddo;  enddo;  enddo
enddo
! invariant: ptr(icell) == cg_start(icell+1) for all cells after Pass 2
```

5. Raytrace Through the Clump Medium

5.1 Overview

Photon propagation is split into two phases:

1. **Advance through vacuum** – use the DDA traversal of the CSR grid (§5.3) to find the next clump hit.
2. **Traverse through a clump** – accumulate optical depth along a straight path through the sphere (§5.4).

Frequency shifts due to individual clump bulk velocities are applied at clump entry and reversed at clump exit.

5.2 Ray–Sphere Intersection

Given a ray $\mathbf{p}(t) = \mathbf{o} + t\hat{\mathbf{k}}$ and a sphere centred at $\mathbf{c}$ with squared radius r_c^2, define $\mathbf{r} = \mathbf{o} - \mathbf{c}$, $b = \mathbf{r} \cdot \hat{\mathbf{k}}$, $\Delta = b^2 - |\mathbf{r}|^2 + r_c^2$. If $\Delta < 0$ there is no intersection. Otherwise

$$t_{\text{entry}} = -b - \sqrt{\Delta}, \qquad t_{\text{exit}} = -b + \sqrt{\Delta}. \tag{13}$$

A hit is recorded when $t_{\text{exit}} > 0$.

5.3 DDA Search for Next Clump

The Amanatides & Woo (1987) digital differential analyser (DDA) traverses the CSR cells along the ray in order of increasing path length. For each cell, all registered clumps are tested with the ray–sphere formula. The traversal stops as soon as the current cell's entry distance exceeds the best hit distance found so far.

```
best_te  = huge;  best_icl = 0
d        = 0     ! path length to current cell entry

do while (.true.)
   if (d > best_te .or. d > t_max) exit
   ! test all clumps in current cell
   do ip = cg_start(icell), cg_start(icell+1)-1
      icl = cg_list(ip)
      if (icl == skip_icl) cycle
      call ray_sphere_isect(..., te, tx2, hit)
      if (hit .and. tx2>0 .and. te<best_te) then
         best_te=te; best_tx2=tx2; best_icl=icl
```

```
      endif
   enddo
   ! advance to next cell (Amanatides & Woo)
   if (tx<=ty .and. tx<=tz) then
      d=tx; ci=ci+si; tx=tx+delx
   elseif (ty<=tz) then
      d=ty; cj=cj+sj; ty=ty+dely
   else
      d=tz; ck=ck+sk; tz=tz+delz
   endif
enddo
```

5.4 Optical Depth Traversal

`raytrace_to_tau_clump` advances the photon until optical depth $\tau_{\rm in}$ is accumulated or the outer sphere is exited.

Inside a clump (`icell_clump` > 0): The remaining path through the current clump is $s = t_{\rm exit}(\mathbf{p}, \hat{\mathbf{k}}, {\rm icl})$. Opacity at the current frequency: $\kappa = \kappa_c\, \phi(x, a_V)$. If $\tau_{\rm rem} \le \kappa s$, scatter at $\mathrm{d}s = \tau_{\rm rem}/\kappa$. Otherwise subtract κs from $\tau_{\rm rem}$, advance to clump exit, shift frequency back to the global frame, and continue in vacuum.

In vacuum (`icell_clump` $= 0$): Call `find_next_clump` (DDA, §5.3) to find the nearest clump. If none is found along the ray, advance to the sphere boundary and exit (`photon%inside` = `.false.`).

Frequency convention: The photon frequency x (in units of $\Delta\nu_D^{cl}$) is maintained in the local clump rest frame while inside a clump, and in the global frame in vacuum:

$$\text{at clump entry:} \quad x \leftarrow x - u_{\rm los}/v_{\rm th}, \tag{14}$$

$$\text{at clump exit:} \quad x \leftarrow x + u_{\rm los}/v_{\rm th}, \tag{15}$$

where $u_{\rm los} = (\mathbf{v}_i \cdot \hat{\mathbf{k}})$.

Jout accumulation: When the photon exits the outer sphere (`photon%inside` becomes `.false.`), the photon weight is accumulated into the escape spectrum:

$$J_{\rm out}[i_x] \mathrel{+}= w, \qquad i_x = \left\lfloor \frac{x - x_{\rm min}}{\delta x} \right\rfloor + 1, \tag{16}$$

where x is the photon frequency (global frame, in units of $\Delta\nu_D$) and w is the photon weight. This mirrors the behaviour of `raytrace_to_tau_car`, where Jout is accumulated upon grid exit. There are three exit paths (clump-then-sphere exit, already-outside-sphere, and no-clump-found), all handled identically.

5.5 Optical Depth to Edge

`raytrace_to_edge_clump` computes τ from the current position to the sphere boundary without moving the photon (read-only input). The same DDA logic is used; frequency shifts at clump entry/exit are tracked locally. This is called by peel-off routines and by the forced-first-scatter initialisation in `run_simulation_mod`.

6. Peel-Off Observations

Peel-off is handled by the standard `peelingoff_rect` routines (`peeling_direct_outside`, `peeling_resonance_nostokes` etc.) with `raytrace_to_edge` pointing to `raytrace_to_edge_clump`.

The observer distance defaults to $d = 100\,R$ and the image pixel size is set so that the full sphere subtends the image:

$$\delta\theta = \frac{1}{N_x/2}\arcsin\left(\frac{R}{d}\right)\ [\mathrm{rad}] \approx \frac{R}{d\,(N_x/2)}\ [\mathrm{rad}]. \tag{17}$$

For $R = 1$, $d = 100$, $N_x = 129$: $\delta\theta \approx 0\overset{\circ}{.}0089$, covering $\pm 0\overset{\circ}{.}57$ (total $1\overset{\circ}{.}14$), which exactly spans the sphere.

7. FITS Output of Clump Information

When `par%save_clump_info = .true.`, the subroutine `write_clumps_fits` (in `clump_mod.f90`) writes a binary-table FITS file `<base>_clumps.fits.gz` containing:

BinTable columns: `X`, `Y`, `Z` (clump centres, code units); `VX`, `VY`, `VZ` (bulk velocities, km/s).

Header keywords (in the BinTable HDU):

Keyword	Value	Description
`N_CLUMPS`	N	number of clumps (realised)
`SPHERE_R`	R	outer sphere radius (code units)
`CL_RAD`	r_c	clump radius (code units)
`F_VOL`	f_{vol}	volume filling factor (realised)
`F_COV`	f_{cov}	covering factor (realised)
`TAU0`	τ_0	line-centre τ per clump
`SIGMA_V`	σ_v	bulk velocity dispersion (km/s)
`TEMP_CL`	T_c	clump temperature used (K)
`RHOKAP`	κ_c	opacity per code unit
`CL_DFREQ`	$\Delta\nu_D^{cl}$	Doppler frequency (Hz)
`VTHERM`	v_{th}	thermal velocity (km/s)
`VOIGT_A`	a_V	Voigt damping parameter
`RMAX`	R	`par%rmax` (input)
`IN_FCOV`	`par%clump_f_cov`	input covering factor
`IN_FVOL`	`par%clump_f_vol`	input filling factor
`IN_NCL`	`par%clump_N_clumps`	input N
`IN_NH`	`par%clump_nH`	input n_H (cm^{-3})
`IN_TEMP`	`par%clump_temperature`	input T_c (K)

Note: column data are written first (which creates the BinTable HDU); header keywords are then written into that same HDU. FITS keyword names must not exceed 8 characters.

8. Input Parameters Summary

Parameter	Default	Description
`par%use_clump_medium`	`.false.`	enable clump mode
`par%rmax`	—	outer sphere radius (code units)
`par%clump_radius`	—	clump radius r_c (code units)
`par%clump_f_cov`	0	covering factor (input)
`par%clump_f_vol`	0	volume filling factor (input)
`par%clump_N_clumps`	0	number of clumps (input, direct)
`par%clump_tau0`	0	line-centre τ per clump
`par%clump_nH`	0	clump n_H (cm^{-3}; requires `distance_unit`)
`par%clump_temperature`	-1	clump T (K); $< 0 \Rightarrow$ `par%temperature`
`par%clump_sigma_v`	0	random velocity σ per component (km/s)
`par%velocity_type`	’’	systematic flow type (see §2.4)
`par%Vexp`	0	expansion speed for `hubble`/`constant_radial` (km/s)
`par%Vx, Vy, Vz`	0	bulk velocity for `parallel_velocity` (km/s)
`par%Vpeak`	0	peak speed for `ssh` (km/s)
`par%rpeak`	0	peak radius for `ssh` (code units)
`par%DeltaV`	0	outer velocity increment for `ssh` (km/s)
`par%Vrot`	0	rotation speed for rotating models (km/s)
`par%rinner`	0	solid-body radius for `rotating_galaxy_halo`
`par%save_clump_info`	`.false.`	write clump FITS file

Example input:

```
&parameters
 par%use_clump_medium = .true.
 par%rmax             = 1.0
 par%clump_radius     = 0.01
 par%clump_f_cov      = 5.0
 par%clump_tau0       = 1000.0
 par%temperature      = 1.0e4
 par%clump_sigma_v    = 0.0
 par%no_photons       = 1e5
 par%spectral_type    = 'voigt'
 par%nxfreq           = 301
 par%velocity_min     = -100.0
 par%velocity_max     =  100.0
 par%nx = 11, par%ny = 11, par%nz = 11
 par%nxim = 129, par%nyim = 129
```

```
 par%save_clump_info   = .true.
 par%out_file          = 'clump_test.fits.gz'
/
```

9. Memory Layout

Clump positions, velocities, and CSR arrays are allocated as MPI-3 shared-memory windows via `create_shared_mem` from `memory_mod`, giving one copy per compute node rather than per MPI rank. On systems with multiple ranks per node this significantly reduces memory usage for large clump populations.
Cleanup is performed by `destroy_clumps` which calls `destroy_shared_mem_all` (collective `MPI_WIN_FREE` across all ranks) followed by explicit `nullify` of all pointers. The individual `destroy_mem` is not used because `MPI_WIN_SHARED_QUERY` returns the rank-0 base address, which differs from the local pointer on non-root ranks.

10. Validation and Examples

The directory `examples/clump_sphere/` contains a suite of six reference runs (plus the legacy `clump_test.in`) covering the main parameter space. All named cases use $R = 1$, $r_c = 0.01$, 10^6 photons, and `par%use_clump_medium = .true.`:

Input file	τ_0	Notes
`clump_tau3_fcov5.in`	10^3	$f_{\rm cov} = 5$, static (reference)
`clump_tau4_fcov5.in`	10^4	$f_{\rm cov} = 5$, static
`clump_tau3_fcov20.in`	10^3	$f_{\rm cov} = 20$, dense covering
`clump_tau3_fcov5_hubble.in`	10^3	$f_{\rm cov} = 5$, Hubble $V_{\rm exp} = 200$ km/s
`clump_tau3_fcov5_sigv.in`	10^3	$f_{\rm cov} = 5$, $\sigma_v = 50$ km/s
`clump_tau3_fcov5_peel.in`	10^3	$f_{\rm cov} = 5$, peel-off 129×129

Run with `bash run.sh` (uses 4 MPI ranks by default) or individually:

```
mpirun -np 4 ../../LaRT.x clump_tau3_fcov5.in
```

Python post-processing:

```
python plot_clump.py          # all runs
python plot_clump.py clump_tau3_fcov5   # single run
```

For the reference case ($f_{\rm cov} = 5$, $\tau_0 = 10^3$):

- Realised $f_{\rm cov} \approx 5.00$, $f_{\rm vol} \approx 0.067$;

- RSA acceptance rate $\sim 74\%$, $N = 66667$ clumps placed;
- Mean scattering count ≈ 6400 per photon;
- Wall time ≈ 3.75 min (10 ranks, 10^5 photons).

The escaped spectrum $J_{\text{out}}(x)$ shows the characteristic double-peak expected for a clumpy Lyman-α source. The peel-off output (`_obs.fits.gz`) contains `Scattered` (HDU 0) and `Direct` (HDU 1) spectral image cubes of shape $(301, 129, 129)$.